Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Projekt 1: Klasyfikacja i regresja z użyciem perceptronu

Przedmiot: Sieci Neuronowe

Autorzy:
Jan Najowski
Stefan Stawicki

Prowadzący:
[Tytuł / Stopień, Imię i Nazwisko]

Warszawa, 19 października 2026

Spis treści

1 Wstęp i cel projektu **1**

2 Implementacja wielowarstwowego perceptronu (MLP) **1**

3 Eksperymenty — Zagadnienie regresji **2**
3.1 Zbiory danych i metodyka badawcza 2
3.2 Wpływ funkcji aktywacji w warstwach ukrytych 2
3.3 Wpływ architektury sieci: głębokość i szerokość 2
3.4 Wpływ miary błędu (funkcji straty) 2
3.5 Wpływ rozmiaru zbioru danych (opcjonalnie) 2
3.6 Podsumowanie i wnioski z części regresyjnej 2

4 Eksperymenty — Zagadnienie klasyfikacji **2**
4.1 Zbiory danych 2
4.2 Eksperyment 1 - Funkcje aktywacji w warstwach ukrytych 3
4.2.1 Zbiór danych simple 3
4.2.2 Zbiór danych three_gauss 5
4.3 Eksperyment 2 - Architektura sieci 7
4.3.1 Zbiór danych simple 7
4.3.2 Zbiór danych three_gauss 11
4.4 Eksperyment 3 - Wpływ funkcji straty 14
4.4.1 Zbiór danych simple 15
4.4.2 Zbiór danych three_gauss 17
4.5 Podsumowanie i wnioski z części klasyfikacyjnej 19

5 Podsumowanie ogólne i wnioski końcowe **19**

1 Wstęp i cel projektu

Celem niniejszego projektu jest samodzielna implementacja wielowarstwowego perceptronu (*Multilayer Perceptron*, MLP) od podstaw z wykorzystaniem wyłącznie biblioteki `NumPy`. W ramach implementacji, przygotowano algorytm wstecznej propagacji błędu oraz stochastycznego spadku gradientu (*Stochastic Gradient Descent*). Przygotowany model posłużył do rozwiązania zagadnień aproksymacji nieliniowej dla funkcji $\mathbb{R} \to \mathbb{R}$ (zadanie regresji) oraz przydzielania klas punktom w przestrzeni $\mathbb{R}^2$ (zadanie klasyfikacji binarnej i wieloklasowej). W ramach pracy zbadano kilka aspektów wpływających na działanie sieci, takich jak dobór funkcji aktywacji (Sigmoid, Tanh, ReLU), głębokości i szerokości architektury (od 0 do 4 warstw ukrytych) oraz zastosowanych miar błędu na wyjściu modelu (MSE, BCE, CCE).

2 Implementacja wielowarstwowego perceptronu (MLP)

Kod źródłowy został podzielony na następujące moduły:

- **layers** — definiuje klasę bazową warstwy oraz warstwę w pełni połączoną (`Dense`), odpowiedzialną za transformację liniową, przechowywanie wag i biasów, a także inicjalizację wag przy pomocy zdefiniowanych rozkładów [1].
- **activations** — zawiera implementacje funkcji aktywacji (`ReLU`, `Sigmoid`, `Tanh`, i `Softmax`) wraz z metodami wyznaczania ich pochodnych na potrzeby propagacji wstecznej.
- **losses** — zawiera implementacje funkcji straty: błąd średniokwadratowy (`MSE`), binarna (`BCE`) i kategoryczna entropia krzyżowa (`CCE`).
- **optimizers** — implementuje algorytm *Stochastic gradient descent*.
- **mlp** — zamyka funckjonalność sieci w klasie `MLP`, przechowując listę warstw i implementując: propagację w przód, propagację wsteczną oraz kroki optymalizatora.
- **utils** — zawiera funkcje pomocnicze, w tym tworzenie mini batch'y, narzędzia do kodowania one-hot, metryki ewaluacyjne oraz ustawianie globalnego ziarna losowości.

Proces uczenia modelu sterowany jest przez metodę `MLP.fit()`, która w trakcie iteracji po kolejnych mini batchach i epokach na bieżąco zapisuje postępy uczenia. Wszystkie metadane zbierane są w formie słownika. Wśród jego kluczy znajdują się m.in. funkcje straty (w skali poszczególnych mini batchy oraz średnie na epokę dla zbioru treningowego i walidacyjnego) wprowadzone metryki (np. *Accuracy* dla klasyfikacji), normy Frobeniusa macierzy wag (dla wizualizacji postępów uczenia) oraz zmiany wartości kilku wybranych z góry wag sieci.

3 Eksperymenty — Zagadnienie regresji

3.1 Zbiory danych i metodyka badawcza

3.2 Wpływ funkcji aktywacji w warstwach ukrytych

3.3 Wpływ architektury sieci: głębokość i szerokość

3.4 Wpływ miary błędu (funkcji straty)

3.5 Wpływ rozmiaru zbioru danych (opcjonalnie)

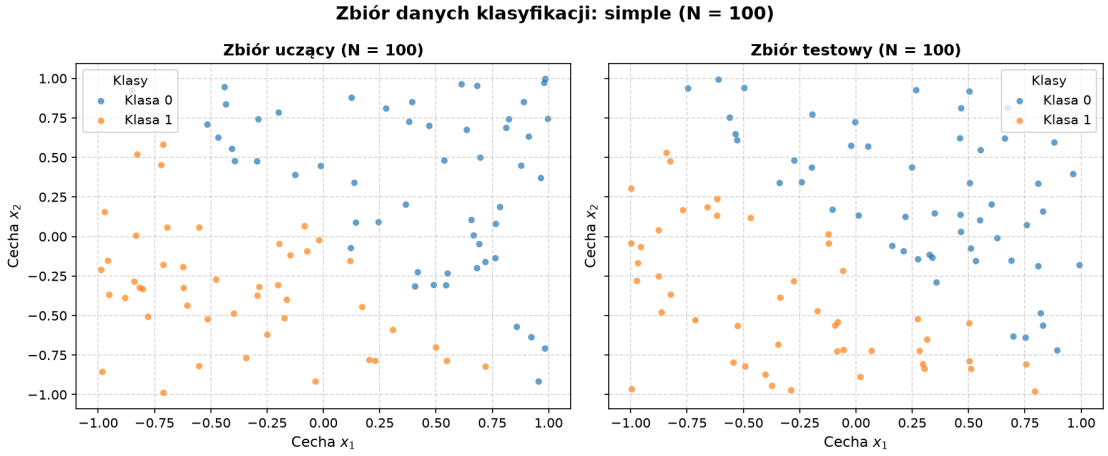
3.6 Podsumowanie i wnioski z części regresyjnej

4 Eksperymenty — Zagadnienie klasyfikacji

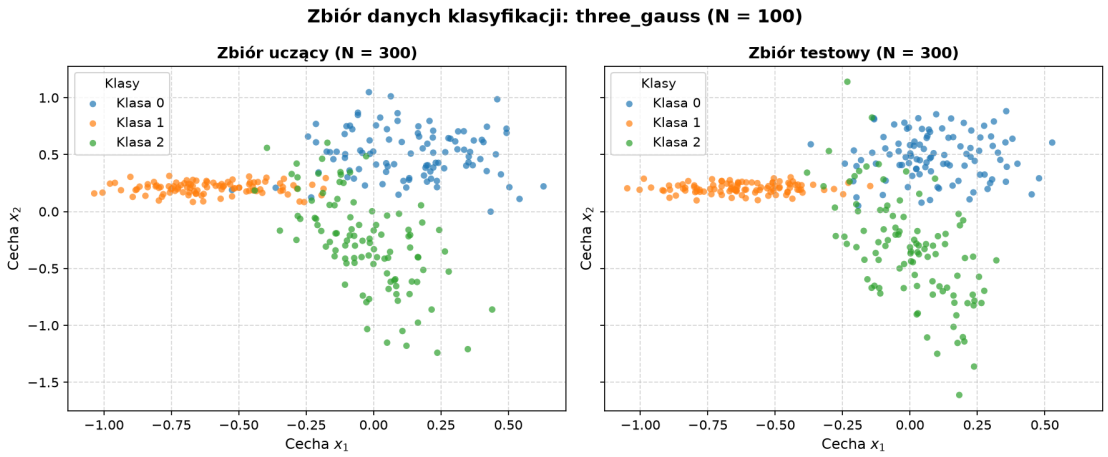
4.1 Zbiory danych

W tej części projektu, badano wykorzystanie modelu `MLP` do klasyfikacji punktów w przestzeni $\mathbb{R}^2$. Do zadania dostarczono dwa zbiory danych, każdy w kilku wersjach różniących się wielkością. Aby uwypuklić różnice między różnymi architekturami, skupiono się na rozmiarze $N = 100$.

- **Zbiór `data.simple`** — klasyfikacja binarnej (2 klasy). Skupiska klas rozdzielone są łagodną, ale nieliniową granicą (Rysunek 1). W warstwie wyjściowej stosowano przeważnie pojedynczy neuron z funkcją aktywacji `Sigmoid` oraz binarną entropię krzyżową (`BinaryCrossEntropyLoss`). Wyjątkiem jest eksperyment 3 którego celem było porównanie entropii krzyżowej do błędu średniokwadratowego (`MSE`).
- **Zbiór `data.three_gauss`** — klasyfikacja wieloklasowa (3 klasy). Skupiska powstały przez nałożenie na siebie dwuwymiarowych rozkładów gaussowskich (Rysunek 2). Kluczową cechą tego zbioru jest wyraźne nakładanie się rozkładów klas na środku przestrzeni. Ze względu na to przenikanie się klas, zbiór jest trudniejszy w klasyfikacji przez co idealne rozpoznawanie klas nie jest możliwe bez przeuczenia. Na wyjściu sieci stosowano 3 neurony z aktywacją `Softmax`, etykiety kodowane w postaci one-hot oraz funkcję kategorycznej entropii krzyżowej (`CategoricalCrossEntropyLoss`). Wyjątkiem ponownie był eksperyment 3 w którym porównano entropię krzyżową z `MSE`.



Rysunek 1: Wizualizacja zbioru uczącego i testowego dla problemu binarnego `data.simple` ($N = 100$).



Rysunek 2: Zbiór uczący i testowy dla problemu wieloklasowego `data.three_gauss` ($N = 100$ na klasę). Widoczne nakładanie się klas w centrum przestrzeni.

4.2 Eksperyment 1 - Funkcje aktywacji w warstwach ukrytych

W ramach eksperymentu zbadano wpływ doboru funkcji aktywacji w warstwach ukrytych (`Sigmoid`, `Tanh`, `ReLU`) na przebieg uczenia, stabilność wag oraz finalny przebieg granic decyzyjnych. Eksperymenty przeprowadzono na jednej architekturze $[2, 32, 16, C]$ ($C = 1$ dla klasyfikacji binarnej, $C = 3$ dla wieloklasowej) przy użyciu optymalizatora `SGD` ($\eta = 0.05$, 150 epok) dla 3 różnych ziaren losowości $(42, 123, 999)$.

Podsumowanie uzyskanych dokładności (*accuracy*) predykcji na zbiorze uczącym i testowym przedstawiono w Tabeli 1.

Tabela 1: Dokładność klasyfikacji dla modeli finalnych w zależności od funkcji aktywacji (średnia $\pm$ odchylenie standardowe z 3 ziaren).

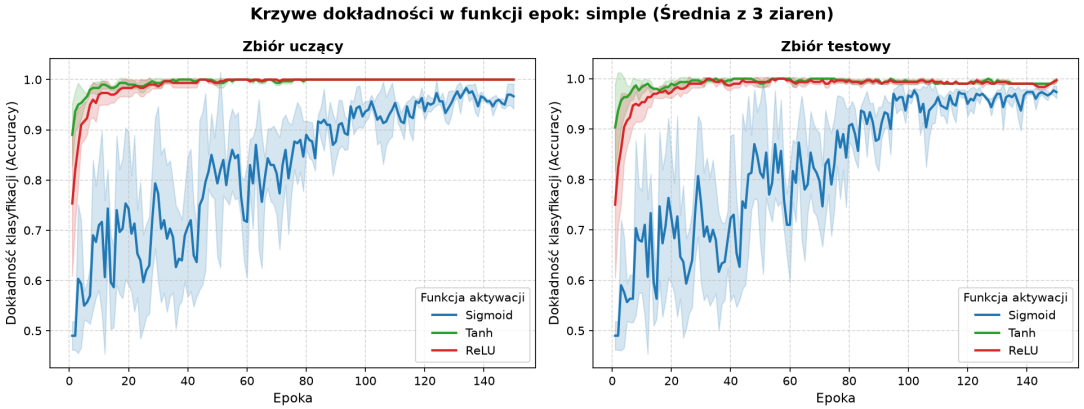
Zbiór danych	Funkcja aktywacji	Zbiór testowy [%]	Zbiór uczący [%]
`data.simple`	Sigmoid	97.33 ± 1.25	96.67 ± 2.49
	Tanh	$\mathbf{99.67} \pm 0.47$	100.00 ± 0.00
	ReLU	$\mathbf{99.67} \pm 0.47$	100.00 ± 0.00
`data.three_gauss`	Sigmoid	93.11 ± 0.16	92.44 ± 0.16
	Tanh	992.8 ± 0.16	92.56 ± 0.16
	ReLU	$\mathbf{93.22} \pm 0.16$	94.11 ± 0.16

4.2.1 Zbiór danych simple

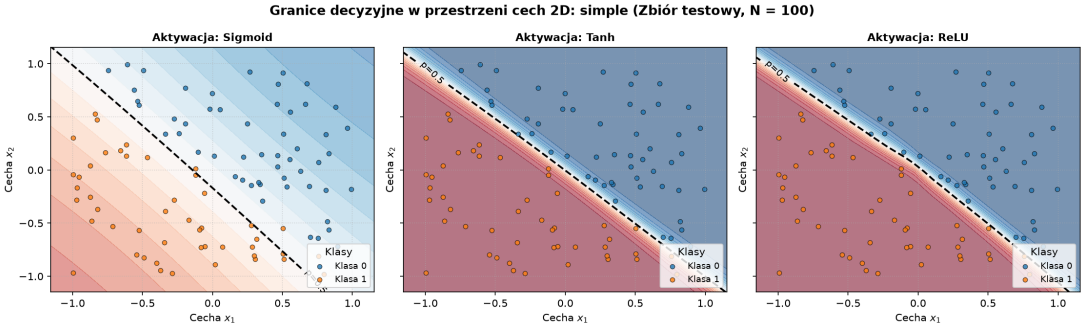
W zadaniu binarnym modele z aktywacją `Tanh` oraz `ReLU` zdołały w 100% nauczyć się rozpoznawać klasy na zbiorach treningowych i dobrze generalizują to na zbiorze testowym (99.67%). Funkcja `Sigmoid` zbiegała wyraźnie wolniej ze względu na nasycenie gradientu (stosunkowo mała wartość pochodnej funkcji sigma). Poniżej przedstawiono wykresy ilustrujące przebieg uczenia i finalne rezultaty na zbiorze prostym.

Na Rysunku 3 można zaobserwować, że dokładność sieci z aktywacją `Sigmoid` znacznie wolniej zbiega do 1. Dodatkowo trajektoria krzywej jest bardziej chaotyczna i wykazuje większe odchylenie standardowe (na badanej próbce 3 ziaren losowych), co sugeruje, że aktywacja `Sigmoid` jest bardziej wrażliwa na losową inicjalizację wag. Gdy porównać nasycenie kolorów na Rysunku 4, widać, że granice decyzyjne dla funkcji `ReLU` i `Tanh` są jasno zdefiniowane, co oznacza, że model jest pewny swoich decyzji. W przypadku `Sigmoid` sytuacja jest odwrotna — występuje szerokie

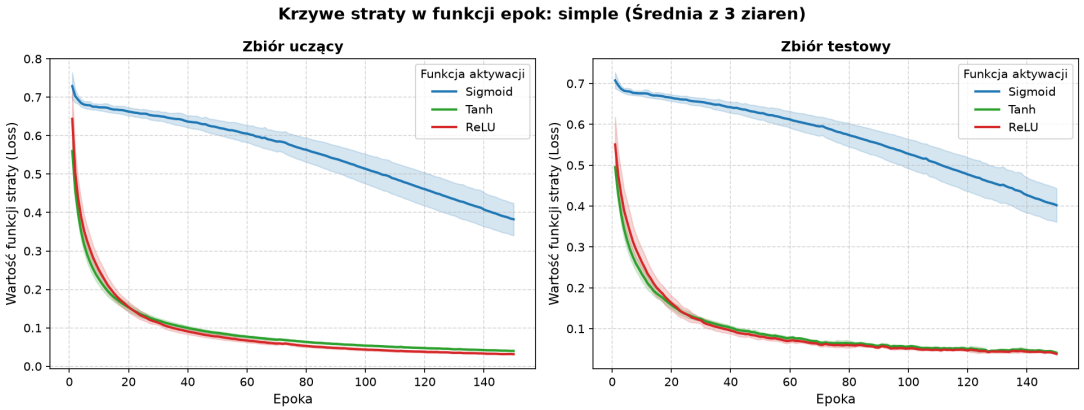
pasmo wysokiej niepewności (szacowane prawdopodobieństwo bliskie 0.5). Przebieg funkcji straty (Rysunek 5) pokazuje gwałtowny spadek błędu treningowego dla `ReLU` i `Tanh` już w pierwszych 20 epokach, podczas gdy strata w modelu `Sigmoid` maleje powoli i jednostajnie przez cały trening. Zmiany pierwszej wagi $w_{0,0}$ oraz biasu b_0 pierwszego neuronu (Rysunek 6) są stabilne i nie ma w nich oscylacji. Zestawienie norm Frobeniusa macierzy wag w każdej z warstw (Rysunek 7) wskazuje na powolny wzrost normy wag we wszystkich warstwach bez nagłych nieprzewidywalnych zmian, przy czym przez większość czasu najwyższe wartości osiąga sieć z aktywacją `ReLU`.



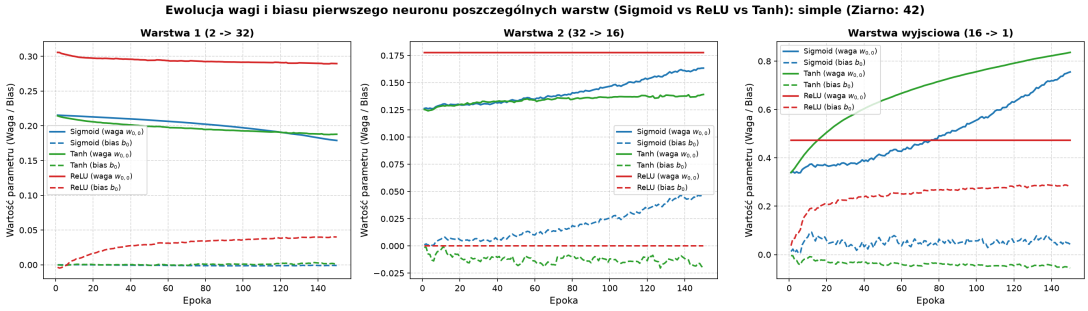
Rysunek 3: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.simple` (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



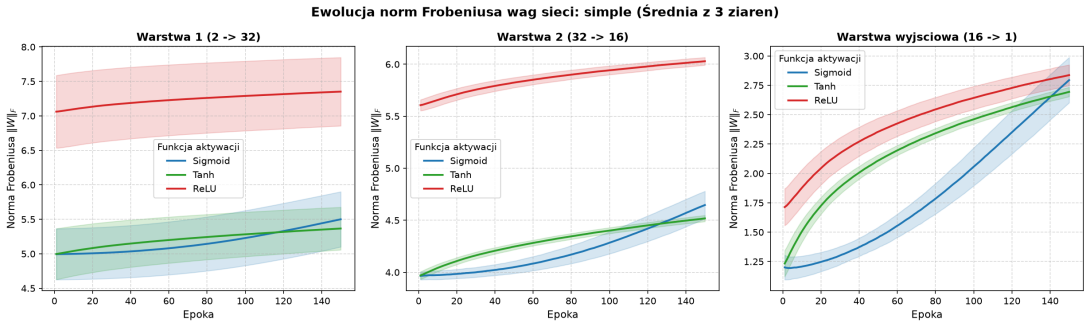
Rysunek 4: Granice decyzyjne w przestrzeni $\mathbb{R}^2$ dla zbioru `data.simple` w zależności od funkcji aktywacji.



Rysunek 5: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.simple` (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 6: Ewolucja wagi $w_{0,0}$ i biasu b_0 pierwszego neuronu poszczególnych warstw dla zbioru `data.simple` (ziarno 42).

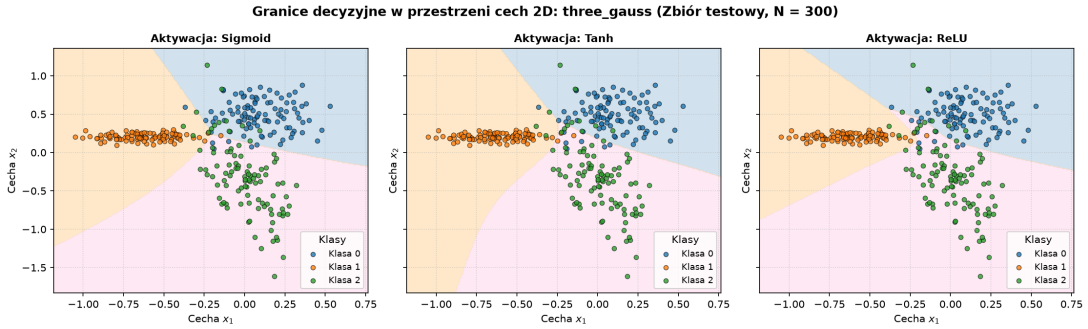


Rysunek 7: Ewolucja norm Frobeniusa wag poszczególnych warstw w trakcie treningu dla zbioru `data.simple`.

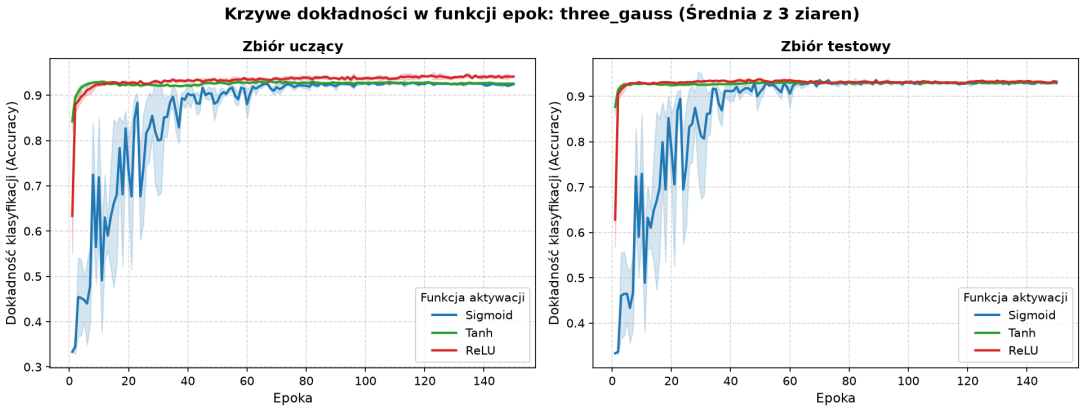
4.2.2 Zbiór danych three_gauss

W zbiorze wieloklasowym wszystkie badane funkcje aktywacji osiągnęły zbliżoną dokładność na poziomie 92.9–93.2%. Ograniczenie to wynika z fizycznego nakładania się na siebie rozkładów trzech klas.

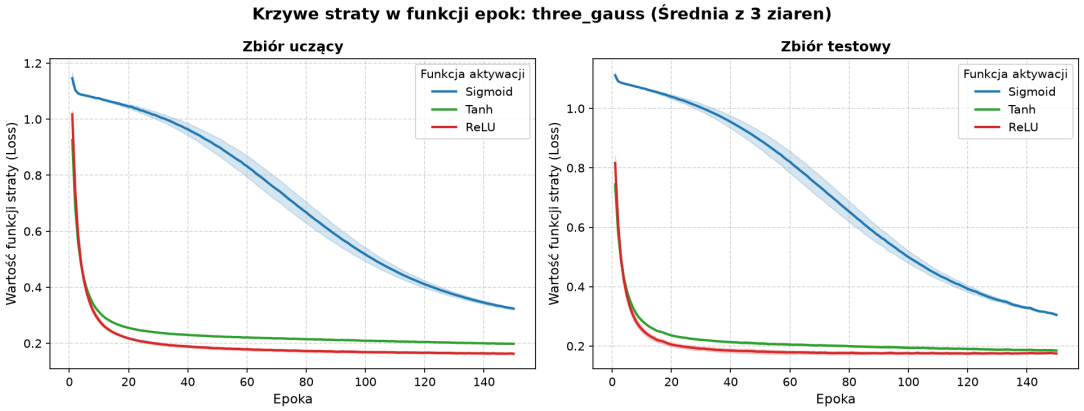
Na Rysunku 8 przedstawiono finalne granice decyzyjne w przestrzeni $\mathbb{R}^2$. Funkcja `ReLU` tworzy odcinkowo liniowe krawędzie podziału, podczas gdy `Tanh` i `Sigmoid` wyznaczają gładkie krzywe. We wszystkich modelach widoczne są punkty błędnie zaklasyfikowane w centralnej strefie nakładania się klas. Krzywe dokładności na Rysunku 9 pokazują, że `ReLU` i `Tanh` już w kilkunastu epokach osiągają dokładność ok. 93%, podczas gdy `Sigmoid` początkowo mocno oscyluje i potrzebuje ponad 80 epok na ustabilizowanie predykcji. Pozostałe wykresy obrazują podobne zjawiska jak w przypadku zbioru `data.simple`.



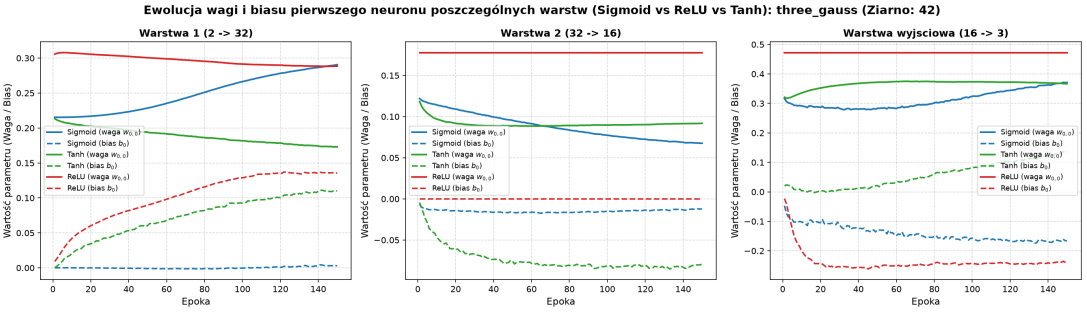
Rysunek 8: Granice decyzyjne w przestrzeni $\mathbb{R}^2$ dla zbioru `data.three_gauss` w zależności od funkcji aktywacji.



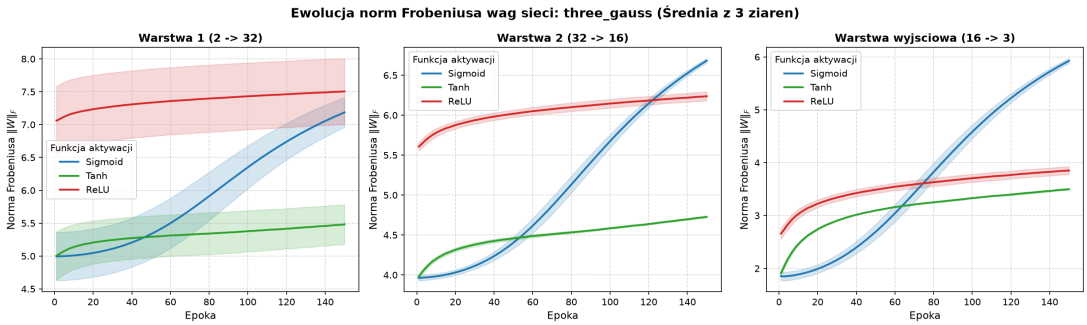
Rysunek 9: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.three_gauss` (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 10: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.three_gauss` (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 11: Ewolucja wagi $w_{0,0}$ i biasu b_0 pierwszego neuronu poszczególnych warstw dla zbioru `data.three_gauss` (ziarno 42).



Rysunek 12: Ewolucja norm Frobeniusa wag poszczególnych warstw w trakcie treningu dla zbioru `data.three_gauss`.

Wnioski. Podsumowując wyniki:

- Funkcje aktywacji `Tanh` oraz `ReLU` zapewniają znacznie szybszą redukcję błędu i wyższą odporność na losową inicjalizację parametrów niż funkcja `Sigmoid`, która cierpi na spowolnienie uczenia wywołane nasyceniem gradientu.
- Kształt wyznaczonych granic decyzyjnych odzwierciedla charakter funkcji aktywacji (odcinkowo liniowe dla `ReLU` vs gładkie dla `Tanh`). W zbiorze `data.three_gauss` w którym klasy się nakładają wszystkie modele osiągają zbliżoną skuteczność ograniczoną górną granicą ok. 93%.
- Ewolucja norm wag oraz trajektorii pojedynczych neuronów przebiega stabilnie we wszystkich wariantach, potwierdzając że implementacja jest poprawna. W kolejnych eksperymentach te wykresy będą pominietę dla większej czytelności.

4.3 Eksperyment 2 - Architektura sieci

W ramach eksperymentu zbadano przebieg uczenia i finalne wyniki sieci w zależności od jej głębokości (od 0 do 4 warstw ukrytych) oraz wielkości poszczególnych warstw. Przetestowano 8 wariantów (gdzie C oznacza liczbę rozpoznawanych klas):

- model liniowy bez warstw ukrytych $[2, C]$
- trzy modele z 1 warstwą ukrytą ($[2, 4, C]$, $[2, 16, C]$, $[2, 64, C]$)
- dwa modele 2 warstwowe ($[2, 16, 8, C]$, $[2, 32, 16, C]$)
- sieć 3 warstwową $[2, 32, 16, 8, C]$
- sieć 4 warstwową $[2, 64, 32, 16, 8, C]$

Na podstawie wyników eskperymentu 1, tym razem zastosowano wszędzie funkcję aktywacji `ReLU` w warstwach ukrytych. Ziarna losowości pozostają bez zmian $(42, 123, 999)$. Natomiast hiperparametry uczenia zostały lekko zmodyfikowane ($\eta = 0.01$, 250 epok) aby głębsze sieci miały szansę zostać dotrenowane.

4.3.1 Zbiór danych simple

Wyniki dokładności klasyfikacji dla badanych architektur na zbiorze `data.simple` przedstawiono w Tabeli 2.

Tabela 2: Dokładność klasyfikacji w zależności od architektury dla zbioru `data.simple` (średnia ± odchylenie standardowe z 3 ziaren).

Architektura	**Warstwy ukryte**	**Zbiór testowy [%]**	**Zbiór uczący [%]**
0 ukrytych: [2]	0	95.33 ± 1.25	94.00 ± 0.82
1 ukryta (4): [2, 4]	1	92.00 ± 3.27	90.00 ± 2.16
1 ukryta (16): [2, 16]	1	98.67 ± 1.25	96.67 ± 0.94
1 ukryta (64): [2, 64]	1	98.33 ± 0.94	99.33 ± 0.47
2 ukryte (A): [2, 16, 8]	2	98.33 ± 0.94	99.33 ± 0.47
2 ukryte (B): [2, 32, 16]	2	$\mathbf{99.33 \pm 0.47}$	100.00 ± 0.00
3 ukryte: [2, 32, 16, 8]	3	98.67 ± 1.25	99.67 ± 0.47
4 ukryte: [2, 64, 32, 16, 8]	4	98.67 ± 0.47	100.00 ± 0.00

Na Rysunku 13 przedstawiono zmiany granic decyzyjnych wraz ze wzrostem złożoności sieci. Model liniowy (zero warstw ukrytych) wyznacza liniową granicę, która nie jest w stanie dokładnie odwzorować zakrzywionego kształtu klas podczas gdy dodanie jednej ukrytej warstwy 16 neuronów pozwala na płynne dopasowanie granicy i znacznie lepsze podzielenie przestrzeni. Głębsze architektury oferują bardziej pewne predykcje i bardziej szczegółową krzywą rozdzielającą przestrzeń.

Krzywe dokładności wraz z postępami w uczeniu (Rysunek 14) pokazują, że sieci głębsze (2-4 warstwy) osiągają zbieżność znacznie szybciej i mają mniejsze odchylenie standardowe. Wąska sieć [2, 4] uczy się z kolei bardzo wolno, z szerokim przedziałem niepewności co sugeruje że jest wrażliwa na inicjalizajcę wag. Podobnie w przypadku funkcji straty, (Rysunek 15) sieci płytkie uczą się wolniej (w szcególności model liniowy przy 250 epokach zatrzymuje się na poziomie ok. 0.38), a dodawanie kolejnych warstw z reguły sprawia że sieć szybciej zmniejsza swój błąd uczenia. Największa przetestowana sieć 4 warstwowa osiąga wartość straty na poziomie 0.05.

Kolejne trzy rysunki przedstawiają zmianę norm wag w czasie uczenia dla kilku architektur. Dla ustalenia uwagi przyglądamy się trzem z nich:

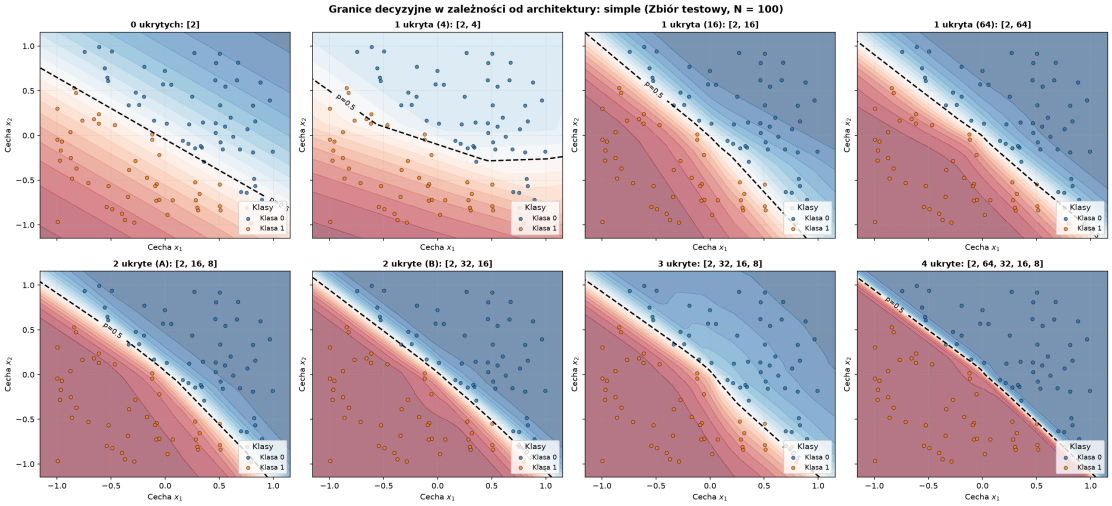
- sieci bardzo płytkiej z 1 warstwą ukrytą: [2, 16, 1] (Rysunek 16),
- sieci średniej z 2 warstwami ukrytymi: [2, 32, 16, 1] (Rysunek 17),
- sieci głębokiej z 4 warstwami ukrytymi: [2, 64, 32, 16, 8, 1] (Rysunek 18).

Porównanie zmian norm Frobeniusa macierzy wag $\|W\|_F$ dla tych trzech sieci prowadzi do następujących obserwacji:

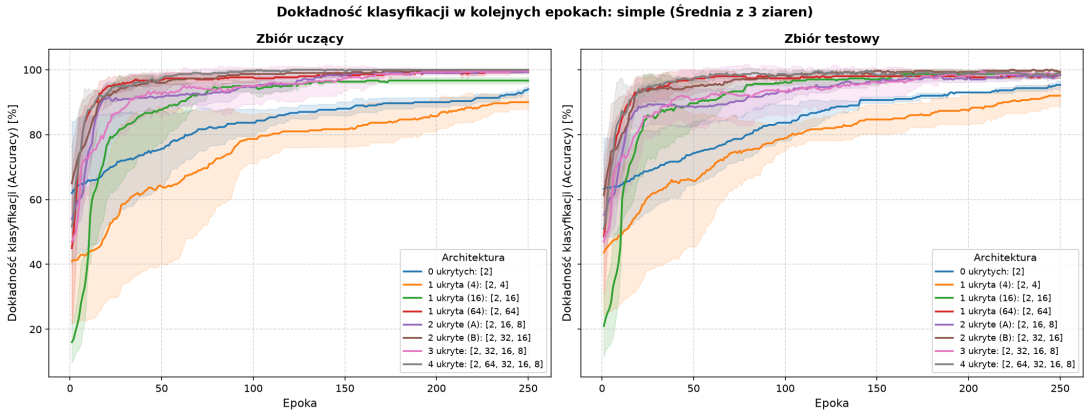
1. **Wartości początkowe**: Wartości początkowe norm zależą od rozmiaru macierzy wag oraz schematu inicjalizacji (`He` dla warstw ukrytych `ReLU`, `Xavier` dla warstwy wyjściowej). Najwyższe normy osiągają macierze o największej liczbie połączeń (np. warstwa 2 o wymiarach 64×32 w sieci 4-warstwowej).

2. **Zmiana parametrów w czasie**: We wszystkich trzech modelach normy wag rosną monotonicznie i gładko w trakcie całego treningu, co świadczy o stabilności aglorytmu `SGD` przy kroku $\eta = 0.01$.

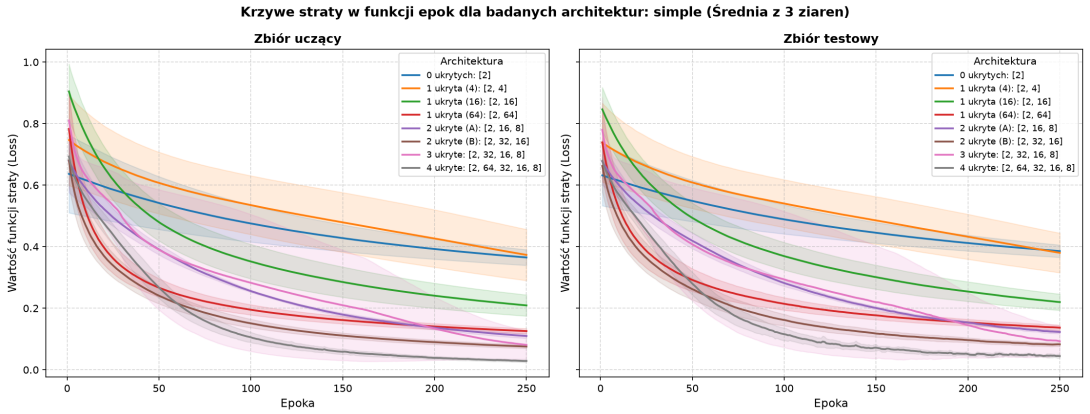
3. **Różnice między warstwami**: W sieci płytkiej [2, 16] norma warstwy wyjściowej rosną najbardziej, prawdopodobnie dlatego, że to ta warstwa dysponuje większą ilością informacji (16 cech zamiast tylko 2). Z kolei w sieciach 2 i 4 warstwowych tempo zmiany norm wag wygląda podobnie.



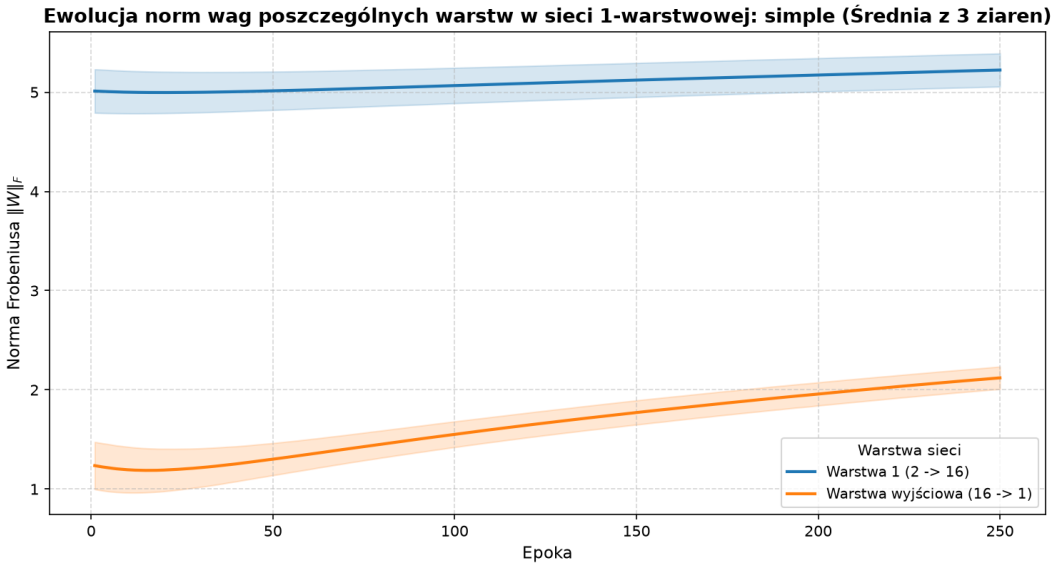
Rysunek 13: Kształt granic decyzyjnych w przestrzeni cech $\mathbb{R}^2$ dla zbioru `data.simple` w zależności od architektury sieci.



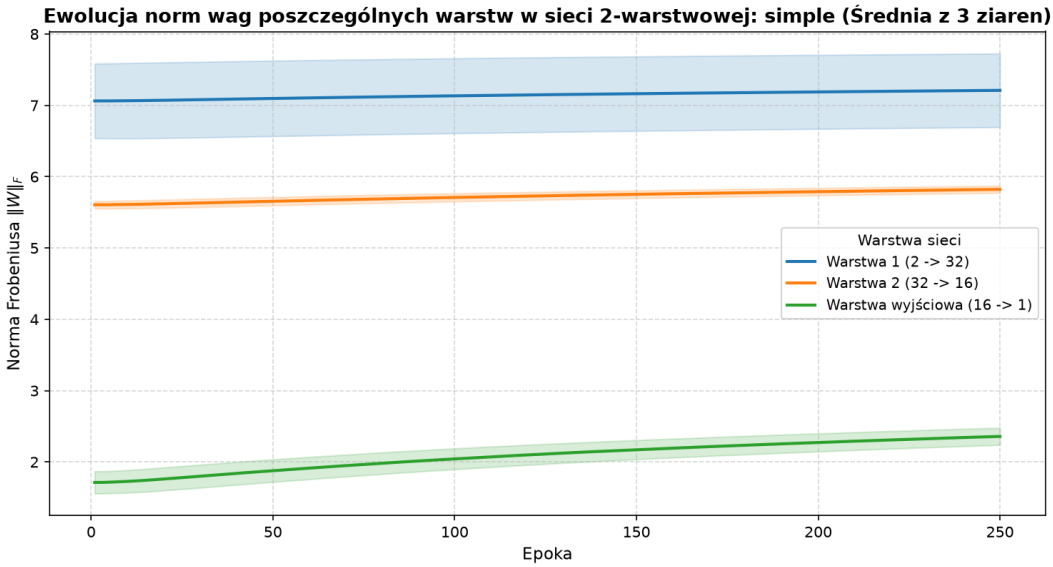
Rysunek 14: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.simple` w zależności od architektury (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



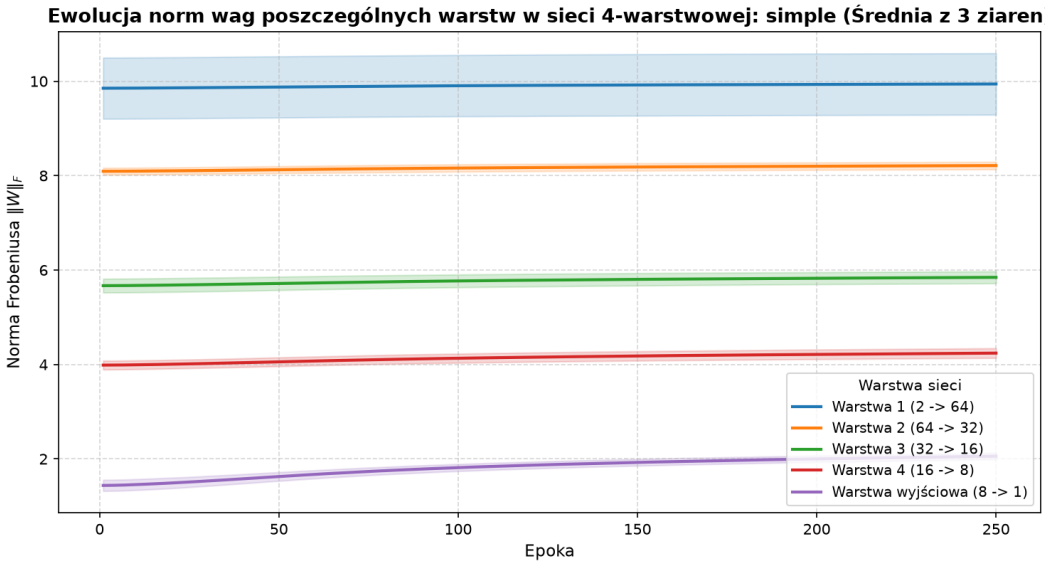
Rysunek 15: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.simple` w zależności od architektury (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 16: Ewolucja norm Frobeniusa wag poszczególnych warstw w sieci 1-warstwowej (`data.simple`, średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 17: Ewolucja norm Frobeniusa wag poszczególnych warstw w sieci 2-warstwowej (`data.simple`, średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 18: Ewolucja norm Frobeniusa wag poszczególnych warstw w sieci 4-warstwowej (`data.simple`, średnia z 3 ziaren z przedziałem $\pm 1\sigma$).

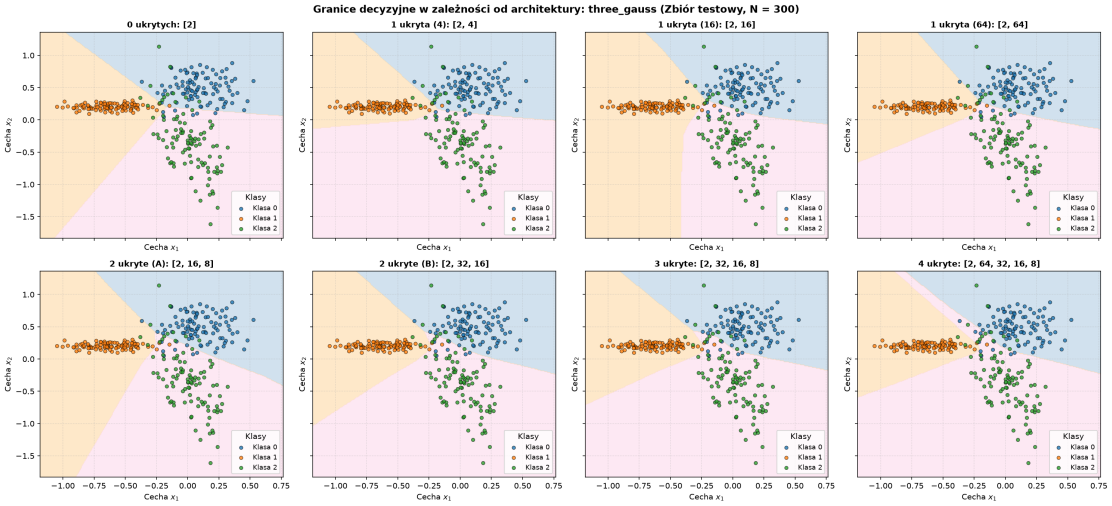
4.3.2 Zbiór danych three_gauss

Wyniki dokładności klasyfikacji w zbiorze wieloklasowym `data.three_gauss` zestawiono w Tabeli 3. Tak jak w eksperymencie 1, można zaobserwować że żaden z modeli nie przekroczył granicy ok. 93.0% z uwagi na nakładające się klasy. Natomiast modele głębsze radziły sobie odrobinę lepiej. Ciekawe jest to, że liniowy model bez żadnych warstw ukrytych osiąga podobny wynik do pozostałych sieci. Wskazuje to na to, że zbiór jest dość łatwy do rozdzielenia w sposób liniowy, a nauczenie się szumu wynikającego z nakładania się klas nie pozwala głębszym modelom osiągnąć znacząco lepszych wyników.

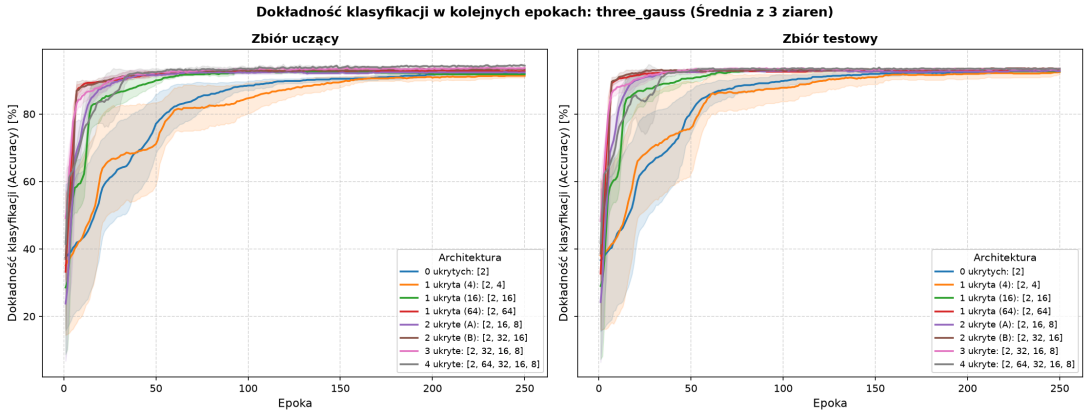
Tabela 3: Dokładność klasyfikacji w zależności od architektury dla zbioru `data.three_gauss` (średnia $\pm$ odchylenie standardowe z 3 ziaren).

Architektura	**Warstwy ukryte**	**Zbiór testowy [%]**	**Zbiór uczący [%]**
0 ukrytych: [2]	0	92.78 ± 0.16	92.00 ± 0.47
1 ukryta (4): [2, 4]	1	92.44 ± 0.87	91.33 ± 0.54
1 ukryta (16): [2, 16]	1	92.67 ± 0.27	91.78 ± 0.16
1 ukryta (64): [2, 64]	1	92.89 ± 0.16	92.67 ± 0.27
2 ukryte (A): [2, 16, 8]	2	92.78 ± 0.16	92.33 ± 0.00
2 ukryte (B): [2, 32, 16]	2	$\mathbf{93.44 \pm 0.16}$	93.22 ± 0.63
3 ukryte: [2, 32, 16, 8]	3	$\mathbf{93.44 \pm 0.16}$	93.67 ± 0.27
4 ukryte: [2, 64, 32, 16, 8]	4	$\mathbf{93.44 \pm 0.16}$	94.33 ± 0.00

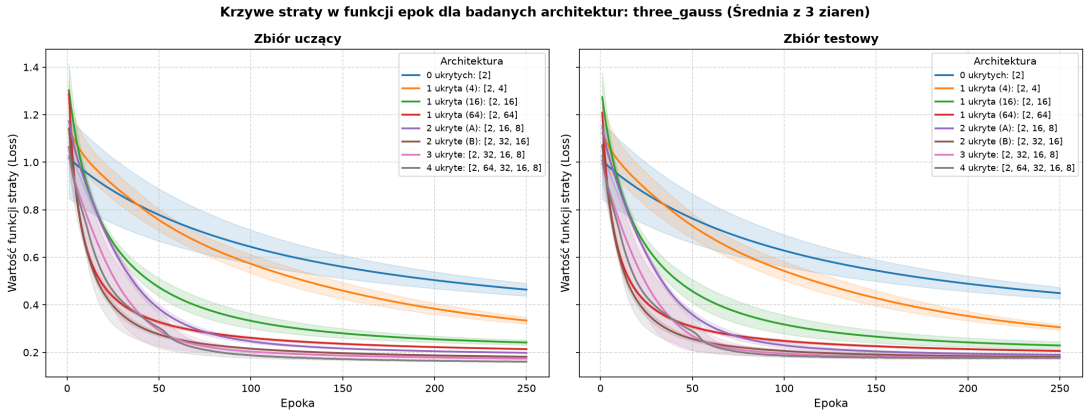
Na Rysunku 19 zobrazowano granice decyzyjne wyznaczone przez badane architektury. Model liniowy, podobnie jak wcześniej dzieli przestrzeń prostymi, podczas gdy architektury głębsze tworzą dokładniejsze granice klasyfikacji. We wszystkich przypadkach jednak w centrum obrazka widoczne są punkty błędnie zaklasyfikowane z uwagi na nakładanie się rozkładów. Pozostałe wykresy prezentują się bardzo podobnie do tych ze zbioru `data.simple`, wnioski są analogiczne.



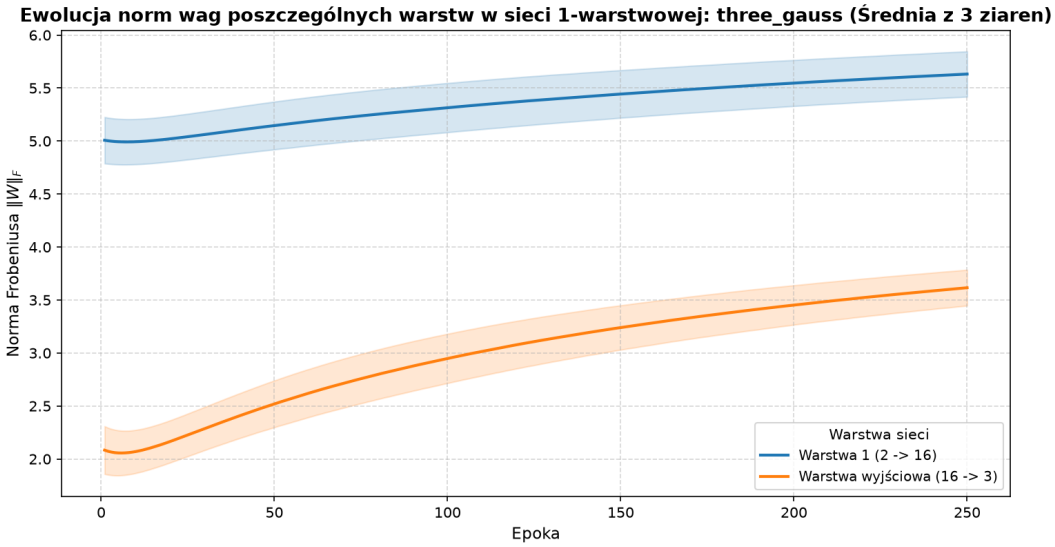
Rysunek 19: Kształt granic decyzyjnych w przestrzeni cech $\mathbb{R}^2$ dla zbioru `data.three_gauss` w zależności od architektury sieci.



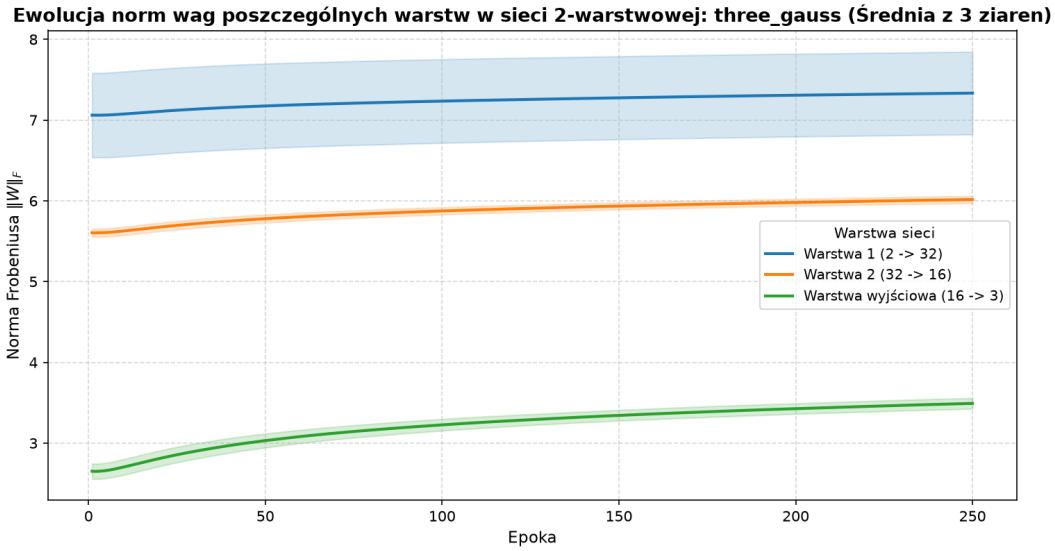
Rysunek 20: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.three_gauss` w zależności od architektury (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



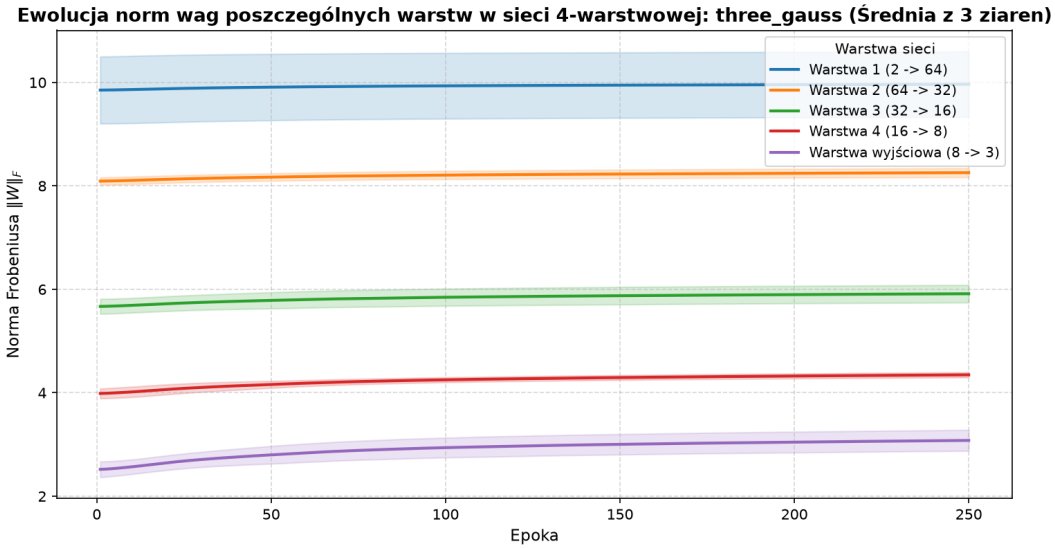
Rysunek 21: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.three_gauss` w zależności od architektury (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 22: Ewolucja norm Frobeniusa wag poszczególnych warstw w sieci 1-warstwowej (`data.three_gauss`, średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 23: Ewolucja norm Frobeniusa wag poszczególnych warstw w sieci 2-warstwowej (`data.three_gauss`, średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 24: Ewolucja norm Frobeniusa wag poszczególnych warstw w sieci 4-warstwowej (`data.three_gauss`, średnia z 3 ziaren z przedziałem $\pm 1\sigma$).

Wnioski. Podsumowując badanie wpływu głębokości i szerokości modelu `MLP`:

- W tych konkretnych zadaniach, model bez warstw ukrytych (liniowy) nieco gorzej od sieci głębszych. Dla zbiory `data.simple` podział jest na tyle prosty, że model liniowy nie popełnia wielu błędów. Z kolei dla `data.three_gauss` szum jest na tyle duży że modele głębokie nie mają szansy daleko prześcignąć modelu liniowego, mimo że uczą się szybciej i osiągają mniejszy błąd.
- Zwiększenie głębokości sieci do 2-4 warstw znacząco przyspiesza tempo zbieżności i obniża błąd treningowy, ale przyrost dokładności powyżej 2 warstw ukrytych jest już mniejszy. Dobrym kompromisem między wielkością sieci i osiąganą dokładnością wydaje się architektura średnia $[2, 32, 16]$. Osiąga ona najwyższy wynik na obu zbiorach (99.33% dla `data.simple` oraz 93.44% dla `data.three_gauss`).
- Ziana norm Frobeniusa wag dla sieci bardzo płytkiej ($[2, 16]$), średniej ($[2, 32, 16]$) oraz głębokiej ($[2, 64, 32, 16, 8]$) potwierdza że optymalizacja wag we wszystkich warstwach jest stabilna. Przy wykorzystaniu aktywacji `ReLU`, inicjalizacja wag He chroni przed eksplozją lub zanikaniem gradientu. W sieci głębokiej tempo zmian wag warstw początkowych jest zbliżone do warstw końcowych (choć nie jest tak dynamiczne jak w poprzednim eksperymencie).
- W zbiorze `data.simple` modele głębokie osiągają niemal zerowe straty treningową bez widocznego spadku wyników na zbiorze testowym. Do tego odchylenia standardowe norm i metryk wskazują na stabilność metody `SGD` względem ziarna losowego.

4.4 Eksperyment 3 - Wpływ funkcji straty

W tym eksperymencie zbadano wpływ doboru funkcji straty na proces uczenia oraz jakość klasyfikacji. Porównano dwie funkcje celu: entropię krzyżową (`Cross-Entropy`: `BinaryCrossEntropy` + aktywacja wyjściowa `Sigmoid` dla klasyfikacji binarnej oraz `CategoricalCrossEntropy` + aktywacja wyjściowa `Softmax` dla wieloklasowej) oraz błąd średniokwadratowy (`MSE`). Pozostałe elementy sieci ustalono na podstawie wyników dwóch poprzednich eksperymentów:

- architektura $[2, 32, 16, \text{wyjście}]$

- warstwy ukryte z aktywacją `ReLU` i inicjalizacją `He`

Aby dokładniej sprawdzić zachowanie miar błedu, przetestowano trzy wartości współczynnika uczenia: $\eta \in \{0.01, 0.05, 0.2\}$. Wyniki uśredniono z 3 ziaren losowych.

4.4.1 Zbiór danych simple

Wyniki dokładności klasyfikacji na zbiorze binarnym `data.simple` zestawiono w Tabeli 4. `Cross-Entropy` osiąga wysoką dokładność już przy najniższym kroku uczenia $\eta = 0.01$ (99.67%). W przypadku błędu średniokwadratowego (`MSE`) przy małym kroku $\eta = 0.01$ sieć radzi sobie zauważalnie słabiej (97.33%). Dopiero zwiększenie kroku uczenia do $\eta = 0.05$ lub $\eta = 0.2$ pozwala modelowi z funkcją `MSE` dorównać wynikom `Cross-Entropy`.

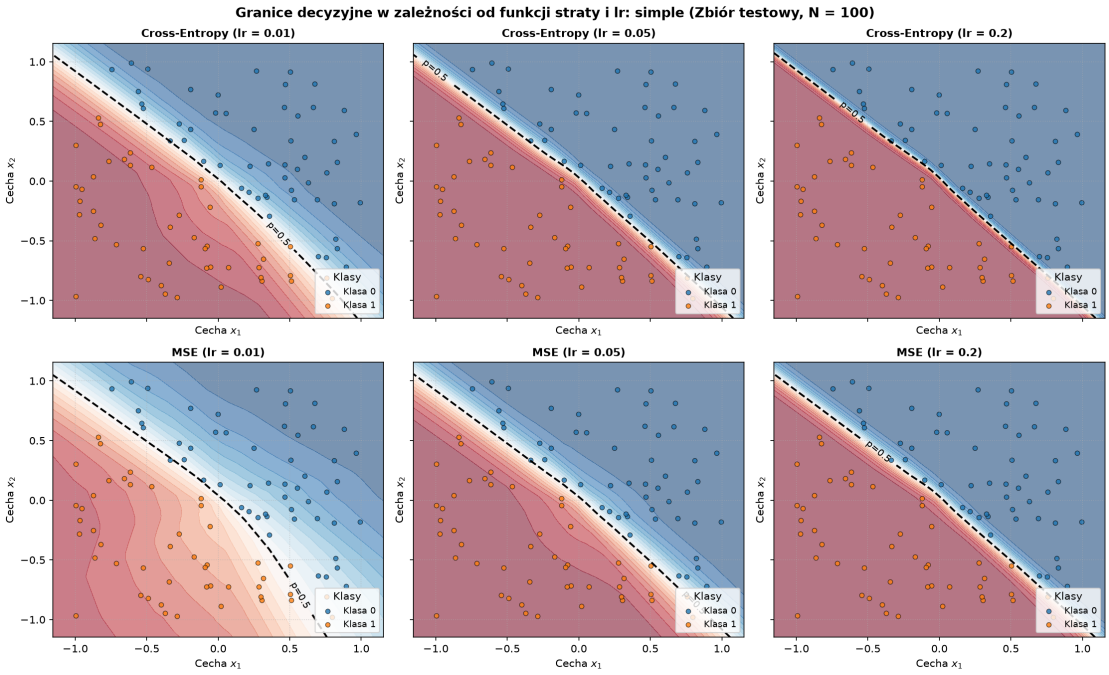
Tabela 4: Dokładność klasyfikacji w zależności od funkcji straty i współczynnika uczenia η dla zbioru `data.simple` (średnia ± odchylenie standardowe z 3 ziaren).

Funkcja straty	**Krok uczenia (η)**	**Zbiór testowy [%]**	**Zbiór uczący [%]**
`Cross-Entropy`	0.01	$\mathbf{99.67 \pm 0.47}$	99.00 ± 1.41
`Cross-Entropy`	0.05	$\mathbf{99.67 \pm 0.47}$	100.00 ± 0.00
`Cross-Entropy`	0.20	99.33 ± 0.47	100.00 ± 0.00
`MSE`	0.01	97.33 ± 1.25	98.00 ± 1.41
`MSE`	0.05	$\mathbf{99.67 \pm 0.47}$	100.00 ± 0.00
`MSE`	0.20	99.33 ± 0.47	100.00 ± 0.00

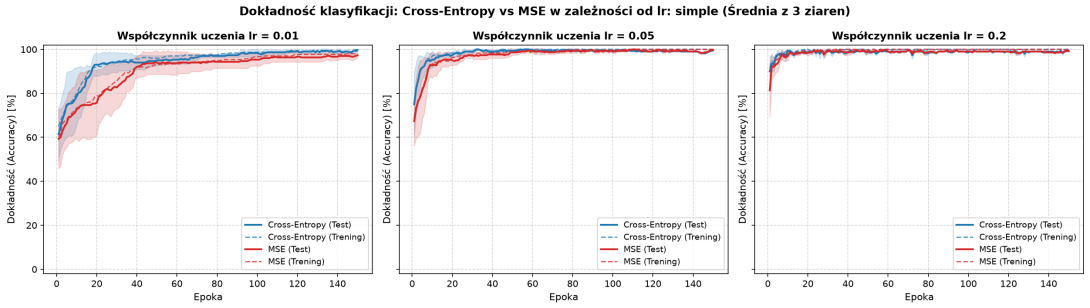
Na Rysunku 25 przedstawiono wyznaczone granice decyzyjne. Obie funkcje straty poprawnie rozdzielają zbiór, jednak dla `MSE` przy $\eta = 0.01$ granica jest mniej gładka i widać większe niezdecydowanie modelu w pobliżu punktów skrajnych. Przy wyższych współczynnikach uczenia granice stają się ostre i bardzo pewne w obu przypadkach.

Przebieg dokładności (Rysunek 26) oraz straty (Rysunek 27) ujawnia wyraźną przewagę tempa uczenia na korzyść `Cross-Entropy`. Model z `Cross-Entropy` osiąga niemal maksymalną dokładność już w ciągu pierwszych kilkunastu epok, podczas gdy `MSE` przy $\eta = 0.01$ uczy się wolno i z większym rozrzutem pomiędzy ziarnami.

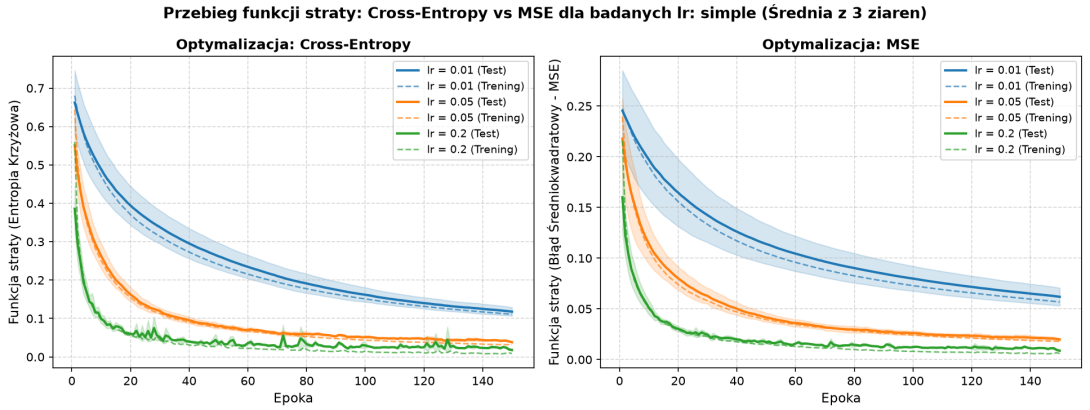
Kluczowe wyjaśnienie tego zachowania widać na wykresie dynamiki aktualizacji wag (Rysunek 28). Dla funkcji `MSE` obserwujemy zjawisko nasycenia gradientu. Pochodna błędu `MSE` względem sygnału wejściowego do neuronu wyjściowego zawiera mnożnik w postaci pochodnej sigmoidy $\sigma'(z)$. Gdy predykcja wyjściowa jest bliska 0 lub 1, pochodna $\sigma'(z)$ dąży do zera, co drastycznie tłumi gradient i sprawia, że poprawki wag $\|\Delta W\|$ są bardzo małe. W przypadku `Cross-Entropy` człon $\sigma'(z)$ skraca się w trakcie wyznaczania gradientu, a sygnał błędu zależy wprost od różnicy $(y_{pred} - y_{true})$, co zapewnia duże i skuteczne aktualizacje od samego początku treningu. Dopiero większy krok uczenia ($\eta = 0.05, 0.2$) w przypadku `MSE` pozwala zrekompensować małe wartości gradientu.



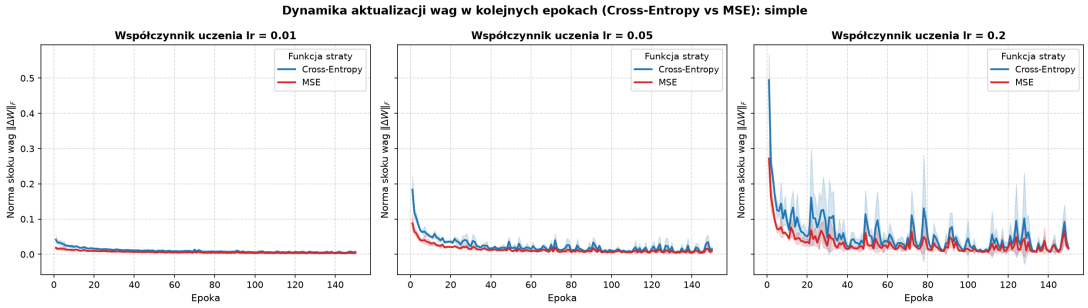
Rysunek 25: Granice decyzyjne w zależności od funkcji straty i kroku uczenia η dla zbioru `data.simple`.



Rysunek 26: Przebieg dokładności klasyfikacji w kolejnych epokach dla funkcji `Cross-Entropy` oraz `MSE` przy różnych wartościach η (`data.simple`, średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 27: Przebieg funkcji straty w kolejnych epokach dla `Cross-Entropy` i `MSE` na rozdzielonych skalach (`data.simple`, średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 28: Dynamika wielkości aktualizacji wag $\|\Delta W\|$ w kolejnych epokach treningu (`data.simple`). Wyraźnie widoczne tłumienie aktualizacji dla `MSE` przy $\eta = 0.01$.

4.4.2 Zbiór danych three_gauss

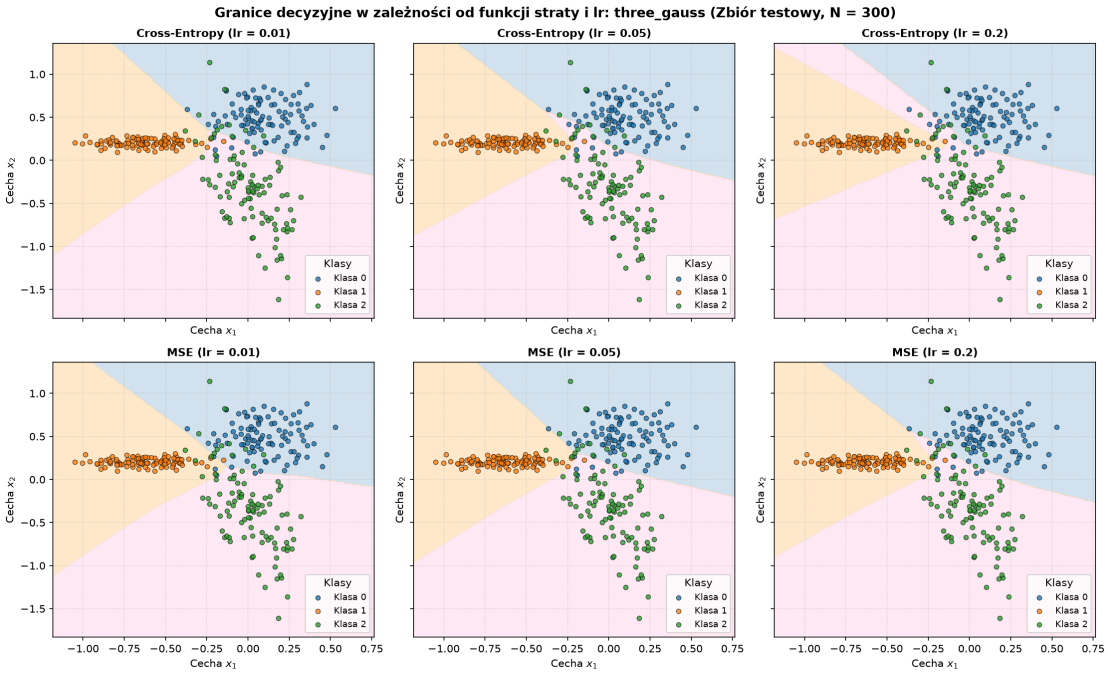
Wyniki dokładności dla wieloklasowego zbioru `data.three_gauss` z warstwą wyjściową `Softmax` przedstawiono w Tabeli 5. Tak jak w poprzednich eksperymentach, ze względu na fizyczne nakładanie się rozkładów gaussowskich, żaden model nie przekracza progu ok. 93.2%.

Nową obserwacją jest to, że w problemie wieloklasowym różnice w końcowej dokładności pomiędzy `Cross-Entropy` a `MSE` są jeszcze mniejsze: zarówno `Cross-Entropy`, jak i `MSE` osiągają ok. $93.0 - 93.2\%$ na zbiorze testowym. Niemniej jednak, przy małym kroku $\eta = 0.01$ funkcja `MSE` uczy się zauważalnie wolniej na zbiorze treningowym (91.44% wobec 92.89% dla `Cross-Entropy`), co wskazuje na występowanie analogicznego zjawiska spłaszczenia gradientu przez pochodną funkcji `Softmax`.

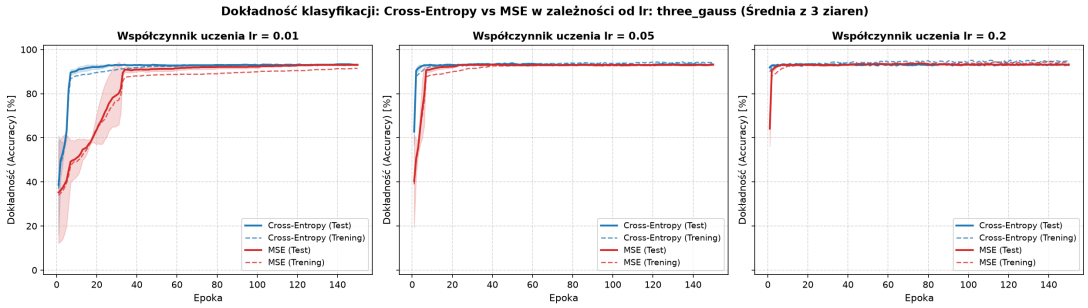
Tabela 5: Dokładność klasyfikacji w zależności od funkcji straty i współczynnika uczenia η dla zbioru `data.three_gauss` (średnia $\pm$ odchylenie standardowe z 3 ziaren).

Funkcja straty	**Krok uczenia (η)**	**Zbiór testowy [%]**	**Zbiór uczący [%]**
`Cross-Entropy`	0.01	93.11 ± 0.16	92.89 ± 0.16
`Cross-Entropy`	0.05	$\mathbf{93.22 \pm 0.16}$	94.11 ± 0.16
`Cross-Entropy`	0.20	$\mathbf{93.22 \pm 0.42}$	94.78 ± 0.57
`MSE`	0.01	93.00 ± 0.00	91.44 ± 1.03
`MSE`	0.05	93.11 ± 0.42	93.00 ± 0.27
`MSE`	0.20	93.00 ± 0.27	93.89 ± 0.42

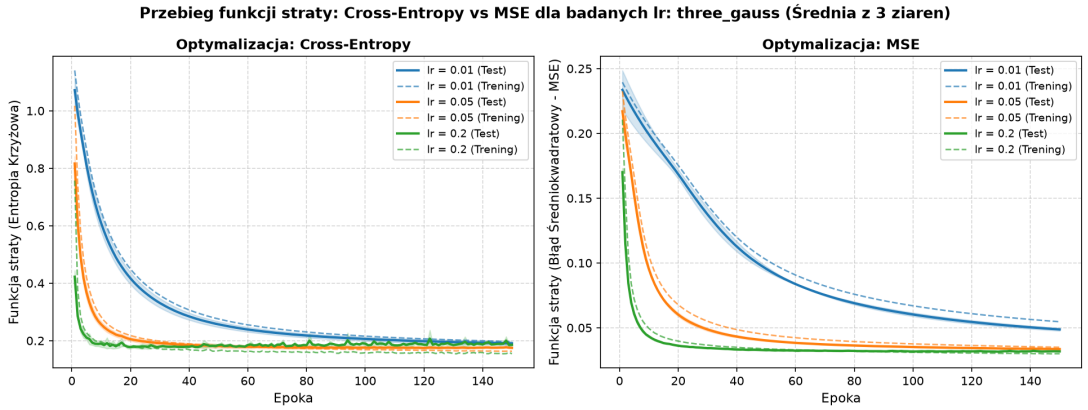
Na Rysunku 29 zobrazowano granice decyzyjne dla badanych wariantów. Dla obu funkcji straty przy krokach $\eta = 0.05$ i $\eta = 0.2$ granice prawidłowo wyodrębniają trzy obszary, a punkty błędnie zaklasyfikowane leżą wyłącznie w rejonie wzajemnego przenikania się klas w centrum. Pozostałe wykresy (Rysunki 30, 31 oraz 32) prezentują zbliżone zjawiska i wnioski będą analogiczne jak dla zbioru `data.simple`.



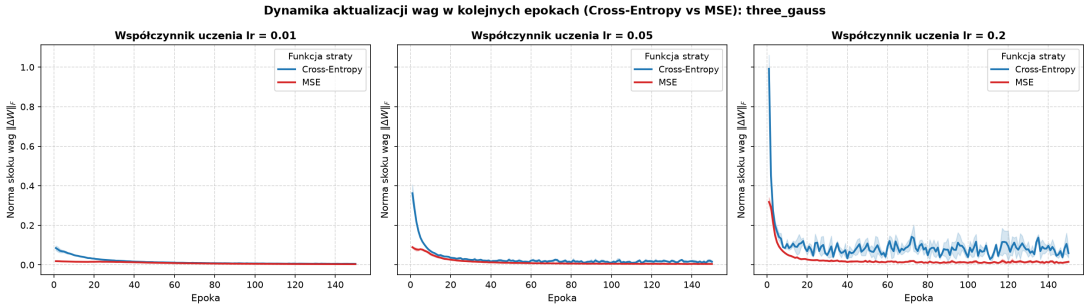
Rysunek 29: Granice decyzyjne w zależności od funkcji straty i kroku uczenia η dla zbioru `data.three_gauss`.



Rysunek 30: Przebieg dokładności klasyfikacji w kolejnych epokach dla zbioru `data.three_gauss` (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 31: Przebieg funkcji straty w kolejnych epokach dla zbioru `data.three_gauss` (średnia z 3 ziaren z przedziałem $\pm 1\sigma$).



Rysunek 32: Dynamika wielkości aktualizacji wag $\|\Delta W\|$ w kolejnych epokach treningu (`data.three_gauss`).

Wnioski. Podsumowując porównanie `Cross-Entropy` oraz błędu średniokwadratowego (`MSE`) w zadaniach klasyfikacji:

- **Przewaga `Cross-Entropy`**: Entropia krzyżowa jest lepiej dopasowana do zadań klasyfikacyjnych. Zapewnia szybszą zbieżność i wysoką dokładność już przy małych wartościach kroku uczenia ($\eta = 0.01$).
- **Zjawisko nasycenia w `MSE`**: Zastosowanie funkcji `MSE` prowadzi do spłaszczenia gradientu w pobliżu nasyconych stanów wyjścia (gdy wyjście jest bliskie 0 lub 1). Obecność pochodnej $\sigma'(z)$ w gradiencie drastycznie zmniejsza poprawki wag $\|\Delta W\|$, przez co sieć uczy się powoli w początkowych fazach.
- **Wpływ współczynnika uczenia**: Wolniejsze uczenie modelu z funkcją `MSE` można częściowo skompensować poprzez zwiększenie wartości η (np. do 0.05 lub 0.2). Przy większym kroku uczenia sieć z funkcją `MSE` osiąga ostatecznie zbliżoną dokładność do modelu z `Cross-Entropy`. Mimo to, entropia krzyżowa ma tę przewagę że nie musimy ręcznie dostrajać współczynnika uczenia.

4.5 Podsumowanie i wnioski z części klasyfikacyjnej

- **Funkcja aktywacji**: Najlepsze rezultaty zapewnia funkcja `ReLU` (w połączeniu z inicjalizacją `He` - zgodnie z [1]). Zabezpiecza ona model przed zanikaniem gradientu i gwarantuje najszybszą zbieżność w porównaniu do nasycającej funkcji funkcji `Sigmoid`.
- **Architektura sieci**: Zwiększenie liczby warstw ukrytych przyspiesza proces uczenia i pozwala wyznaczać dokładniejsze granice decyzyjne. Najlepszym kompromisem pomiędzy rozmiarem sieci a dokładnością okazała się architektura 2-warstwowa ($[2, 32, 16]$) — dalsze pogłębianie modelu przynosiło niewielki zysk na zbiorze testowym.
- **Funkcja straty**: Entropia krzyżowa (`Cross-Entropy`) jest lepszym wyborem niż `MSE` dla problemu klasyfikacji. Eliminuje problem nasycenia gradientu na wyjściu z aktywacją `Sigmoid` lub `Softmax`, co zapewnia szybkie i stabilne uczenie już przy małym kroku η.

5 Podsumowanie ogólne i wnioski końcowe

Literatura

[1] The Deep Hub, *He and Xavier Weight Initialization Functions*, `https://medium.com/thedeephub/he-and-xavier-weight-initialization-functions-acedc5322ce5`.